INFORMATICS INSTITUTE OF TECHNOLOGY

In Collaboration With

UNIVERSITY OF WESTMINSTER

Module : Client Server Architecture

5COSC022C.2

Date : April 2026

REST API design, development and implementation - Coursework Report

Student Name	UOW ID	IIT ID
H M C S Herath	W2153371	20240408

PART 1: SETUP & DISCOVERY

1.1 Architecture & Configuration: Resource Lifecycle & Synchronisation

In JAX-RS implementations such as Jersey, resource classes—including `RoomResource`, `SensorResource`, and `DiscoveryResource`—are request-scoped by default. This architecture ensures that a new, independent instance of the resource class is instantiated for every incoming HTTP request and subsequently destroyed once the response is dispatched. This design choice inherently guarantees thread-safety for instance variables declared within the resource class, as they are never shared between concurrent requests.

However, our API utilizes `ResourceConfig.forApplication(new ApiApplication())` for bootstrapping and relies on an `InMemoryStore` to persist data across multiple requests. To facilitate this persistence, the `InMemoryStore` is implemented as a Singleton. Consequently, multiple request-scoped resource threads concurrently access and mutate this single, shared state. To prevent critical race conditions and maintain data integrity, the following thread-safety mechanisms are employed:

- **Thread-Safe Collections:** The store utilizes `ConcurrentHashMap` for rooms and sensors, ensuring atomic operations for basic data retrieval and updates. Additionally, `Collections.synchronizedList` is used for sensor readings to prevent data corruption during concurrent additions.
- **Synchronized Blocks:** When performing compound mutations involving multiple objects—such as adding a new sensor while simultaneously updating the parent room's `sensorIds` list within `SensorResource.java`—we utilize explicit `synchronized (room)` blocks. This ensures that the relationship between the two entities remains consistent and accurate.

Trade-offs: While in-memory synchronization is efficient and suitable for this coursework, it lacks scalability in distributed, multi-server RESTful architectures (horizontal scaling). In a production environment, state management would typically be delegated to a distributed database system, such as PostgreSQL or Redis, which handles ACID transactions and concurrency control inherently. This allows the REST APIs to remain purely stateless.

1.2 Discovery Endpoint: Exceptional Justification of HATEOAS

HATEOAS (Hypermedia As The Engine Of Application State) represents a fundamental shift in how clients interact with an API. While traditional static documentation (such as PDFs or Swagger UI) requires developers to hardcode specific endpoint URIs, HATEOAS enables the API to be self-documenting and dynamically discoverable.

By returning resource maps in our `/api/v1` discovery endpoint, we achieve several significant advantages:

- **Reduced Client Coupling:** Clients do not require prior knowledge of exact URI structures. They only need the primary entry point (`/api/v1`) and the semantic relationships (e.g., following the "rooms" link).
- **API Evolution and Maintenance:** If the server needs to restructure its endpoints (for instance, migrating `/rooms` to a new microservice at `/facilities/rooms`), static documentation would immediately fail for existing clients. With HATEOAS, the server simply updates the link returned in the discovery payload, and the client follows the new path without requiring code changes.
- **State-Driven Discoverability:** HATEOAS can dictate the application state dynamically. For example, a client may only receive a "delete" link in a response if they possess the necessary permissions to delete that resource, preventing unauthorized or invalid operations.

PART 2: ROOM MANAGEMENT

2.1 Room Implementation: Analysis of ID-only vs. Full-Object Returns

When designing list endpoints like `GET /rooms`, architects must balance the trade-offs between returning a list of full objects versus a list of identifiers (IDs) or partial objects.

- **Full-Object Returns (Current Implementation):** Returning the complete `Room` object is highly efficient for client-side rendering. It eliminates the need for secondary fetches, providing a smoother initial user experience and reducing the complexity of client-side logic. However, as datasets grow, this approach significantly increases payload overhead, network bandwidth consumption, and server-side serialization time.
- **ID-only Returns:** Returning only IDs is lightweight and fast. However, if a client needs to display a dashboard of rooms, it forces the client to iterate over the IDs and make

subsequent `GET /rooms/{id}` requests. This leads to the "N+1 query" problem, resulting in high network latency and excessive HTTP connection overhead.

Optimization Strategies: For a scalable API, neither extreme is ideal. Superior strategies include:

1. **Field Filtering (Sparse Fieldsets):** Allowing the client to request only necessary fields via query parameters, such as `GET /rooms?fields=id,name`.
2. **Pagination:** Limiting the number of objects returned per request (e.g., `GET /rooms?offset=0&limit=10`) ensures that payload size remains manageable while still delivering sufficient data to render a single view.

2.2 Deletion & Logic: Justification of Idempotency

In REST principles, an HTTP method is considered idempotent if the intended effect on the server of multiple identical requests is the same as the effect of a single request. The `DELETE /rooms/{roomId}` operation is designed to be strictly idempotent.

If a client successfully calls `DELETE` for a specific room, the room is removed. If a network timeout occurs and the client is uncertain if the request succeeded, they can safely retry the same `DELETE` request. On the second attempt, the server will return a `404 Not Found` (or a similar status), but crucially, the underlying server state—the absence of that specific room—remains identical to the state after the first call. No further side effects, such as deleting other rooms or corrupting data, occur. This predictable behavior is essential for reliability, allowing distributed systems to safely implement automatic retry mechanisms.

PART 3: SENSORS & FILTERING

3.1 Sensor Integrity: 415 Unsupported Media Type Consequences

JAX-RS utilizes the `@Consumes` annotation (e.g., `@Consumes(MediaType.APPLICATION_JSON)`) to define the precise MIME type an endpoint expects. While our application logic uses `422 Unprocessable Entity` for semantic payload errors, JAX-RS provides a primary line of defense using `415 Unsupported Media Type` for syntactical content-type mismatches.

If a client attempts to send an XML payload or `text/plain` data to the `POST /sensors` endpoint, the JAX-RS framework intercepts the request before it reaches our Java method. It immediately rejects the request with a `415` status code. This has two major technical consequences:

1. **Fail-Fast Security:** It prevents the server's JSON parsers from attempting to deserialize malformed or maliciously crafted payloads in unsupported formats, protecting the system against parsing vulnerabilities.
2. **Resource Efficiency:** CPU cycles and memory are conserved because application-level validation and routing are completely bypassed for invalid media types.

3.2 Filtered Retrieval: QueryParams vs. PathParams Contrast

RESTful API design draws a sharp contrast between `PathParams` and `QueryParams`:

- **PathParams (`/sensors/{id}`):** These are strictly hierarchical and are used to identify a specific, unique resource or navigate down a defined organizational tree.
- **QueryParams (`/sensors?type=temperature`):** These are non-hierarchical modifiers used to filter, sort, search, or paginate an existing collection of resources.

Query strings are superior for collection filtering because they offer flexibility without altering the fundamental URI structure. Filters are inherently optional and combinable. If paths were used for filtering (e.g., `/sensors/type/temperature/status/active`), the URI structure would become rigid and deeply nested, making it difficult for clients to filter by a single attribute like "status." `QueryParams` elegantly solve this by allowing dynamic, multi-faceted search capabilities while maintaining a pristine base collection URI.

PART 4: SUB-RESOURCES

4.1 Sub-Resource Locator: Managing Complexity & Delegation

As enterprise APIs scale, monolithic controllers that handle all operations for multiple related entities can become unmaintainable. The Sub-Resource Locator pattern (`@Path("/{sensorId}/readings")` returning a new instance of `SensorReadingResource`) is a powerful architectural solution for managing this complexity.

By utilizing this pattern, the API is broken down logically. `SensorResource` no longer manages the CRUD logic for readings; instead, it acts as a router, extracting the `sensorId` context and delegating responsibilities to the dedicated `SensorReadingResource` class. This achieves:

1. **Single Responsibility Principle:** Code organization and maintainability are significantly improved.
2. **Intuitive URI Structure:** It enforces a logical, nested URI hierarchy (`/sensors/{id}/readings`) that accurately reflects the parent-child relationship between entities.
3. **Scalability:** Independent resource classes can be modified, tested, and refactored in isolation without risking regressions in unrelated endpoints.

4.2 Historical Management: Atomicity and Consistency

When `POST /sensors/{id}/readings` is invoked, it is imperative that the new reading is added and the parent sensor's `currentValue` is updated flawlessly. In a highly concurrent environment, such as a smart campus, this operation poses a risk of race conditions.

While `Sensor` objects are stored in a thread-safe `ConcurrentHashMap`, directly modifying an object retrieved from that map is not inherently atomic. If two threads process readings for the same sensor simultaneously, the final `currentValue` might not reflect the chronologically latest reading due to interleaved execution. To guarantee a consistent update, this side-effect operation must be synchronized. By wrapping the update logic within a `synchronized (sensor)` block, we ensure that the recording of the reading and the update of the parent's state are executed atomically.

PART 5: ERROR HANDLING

5.1 Specific Exceptions: Analysis of 422 vs. 404

When processing a valid JSON payload that contains a logical error—such as referencing a non-existent `roomId`—choosing the correct HTTP status code is vital for the developer experience.

Returning a `404 Not Found` in this scenario is semantically incorrect, as `404` indicates the targeted URI (`/sensors`) does not exist. The superior standard is `422 Unprocessable`

`Entity`. This status communicates that the server understood the content type and the syntax was valid, but was unable to process the instructions due to a semantic error. This distinction prevents clients from debugging URI paths unnecessarily and directs attention to the flawed data within the request body.

5.2 Global Safety Net: Cybersecurity Risks of Stack Traces

In production environments, unhandled exceptions frequently generate verbose stack traces. Without a global `ExceptionMapper<Throwable>`, the web server may default to returning this raw stack trace to the client. This represents a critical cybersecurity vulnerability known as Information Disclosure. A stack trace provides an attacker with:

1. **Reconnaissance:** Exposure of the framework, underlying operating system file paths, and precise library versions.
2. **Targeted Exploitation:** Attackers can cross-reference library versions against known CVE databases to launch specific exploits.
3. **Reverse Engineering:** The trace reveals class names and logic flow, allowing attackers to map out internal business logic.

By intercepting unexpected errors and returning a sanitized `500 Internal Server Error` JSON response, we create a secure API that denies malicious actors vital reconnaissance data while logging the actual trace securely on the server.